

A 16-bit x 16-bit MAC Design Using Fast 5:2 Compressors

Ohsang Kwon*, Kevin Nowka**, Earl E. Swartzlander*

**University of Texas at Austin, Texas*

***IBM Ausitn Research Lab, Austin, Texas*

kos@mail.utexas.edu

Abstract

3:2 counters and/or 4:2 compressors have been widely used for multiplier implementations. In this paper, a new logical decomposition is derived for fast 5:2 compressor and is proposed to be used for 16-bit x 16-bit MAC designs. In addition, when the accumulator output is in carry-save form, one row in partial product matrix can be eliminated prior to the partial product reduction process. These new methods are combined and explained with 16-bit x 16-bit 2's complement MAC(multiply and accumulate) designs. The use of the new 5:2 compressor leads to 14% speed improvement in the MAC design over the conventional designs using 4:2 compressors and 3:2 counters.

1: Introduction

Multiplication is one of the most frequently encountered operations in microprocessors and signal processing applications. Due to the delay and complexity of multipliers, efficient designs have been pursued over the last several decades. In this paper, a new design method is proposed for multiplier, multiple addition and especially fused MAC (multiply and accumulate) designs.

The fast multiplication process consists of 3 steps[1][2]: *partial product generation*, *partial product reduction* and *final carry-propagating addition*. To reduce the number of partial products, recoding techniques have been widely used[3][4][5]. In the partial product reduction step, 3:2 counter and/or 4:2 compressor have been basic elements to reduce partial products matrix into 2 rows[1][2][6][7]. The reduction process is managed with CSA (Carry-Save-Adder) array using those basic elements. A high-speed adder is used to get the final result from the 2 rows.

In this paper, a new logical decomposition of 5:2 compressor is derived and proposed to be used for fast multiplication or multiple addition applications. A 16-bit x 16-bit MAC(multiply and accumulate) design is investigated as a design example using the 5:2 compressor and radix-4 Booth recoding technique.

For higher throughput of the MAC operation, the result of partial product reduction can be fed back directly to the next cycle in a redundant form (carry-save). A new method is proposed to eliminate one row of partial product matrix before the actual reduction process begins, which can eliminate one 3:2 counter delay at some specific operand sizes. This method is explained with the 16-bit x 16-bit 2's complement MAC design, combined with the use of the new 5:2 compressor.

In this paper, the new logical decomposition of the 5:2 compressor is derived in section 2. Section 3 explains 2 design examples for the 16-bit x 16-bit 2's complement MAC implementation. Section 4 will discuss possible extensions of the new methods. Finally, a conclusion will be given in section 5.

2: A new logical decomposition of 5:2 compressor

2.1: 3:2 counter and 4:2 compressor

Figure 1 shows block diagrams of 3:2 counter. Its usage in multiplier implementation has been found from the early efforts[1][2] to more recent works in the form of CMOS circuits[8][9][10]. It has 3 input bits of equal weight and 2 output bits, sum and carry, of the same and one greater binary bit weight, respectively. A 3:2 counter takes 2 XOR delays (1 XOR delay is denoted Δ_{xor} , thereafter).

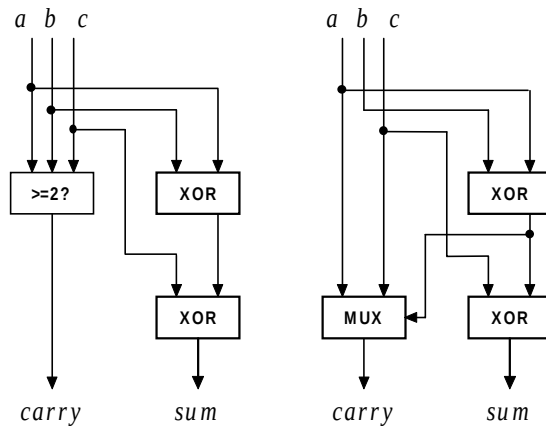


Figure 1. A 3:2 counter

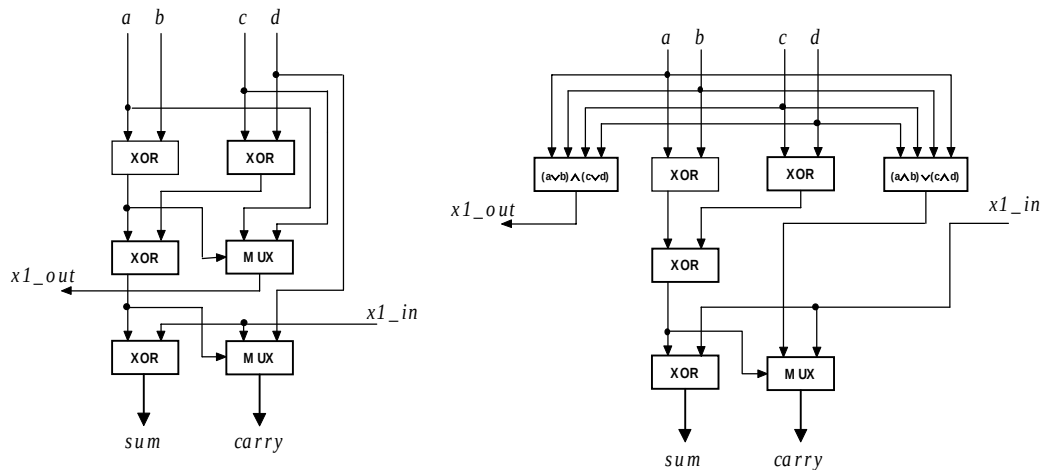


Figure 2. 4:2 compressor logical decompositions

Here, XOR delay is used as speed measure of counters, compressors and partial product reduction throughout this paper since many popular logical decompositions of 3:2 counter and 4:2 compressor have a series of XORs on their critical path. In CMOS technology, XORs and multiplexors(MUX) can be efficiently implemented[7][17]. In this way, the speed comparison can be made independent of specific technologies though actual relative delays may be slightly different depending on specific circuit families and fabrication technologies employed.

Figure 2 shows logical decompositions of 4:2 compressor[7][10][11][12][13]. In the block diagram, ‘ $\vee$ ’ denotes a logical ‘OR’ operation and ‘ $\wedge$ ’ denotes a logical ‘AND’ operation, respectively. The 4:2 compressor has 5 input bits including a carry-in from the neighboring cell of one binary bit lower significance and 3 outputs including a carry-out to the one greater significant cell. Both decompositions of the figure have $3\Delta_{xor}$ on their critical path.

2.2: 5:2 compressor

Figure 3 shows a block diagram of the new 5:2 compressor the delay of which is $4\Delta_{xor}$. It has 5 direct input bits and 2 additional carry-in bits from a neighboring one-lower significant cell. Among 4 output bits, 2 are carry-out bits to the one-greater significant cell and one carry of one-greater significance and a sum bit.

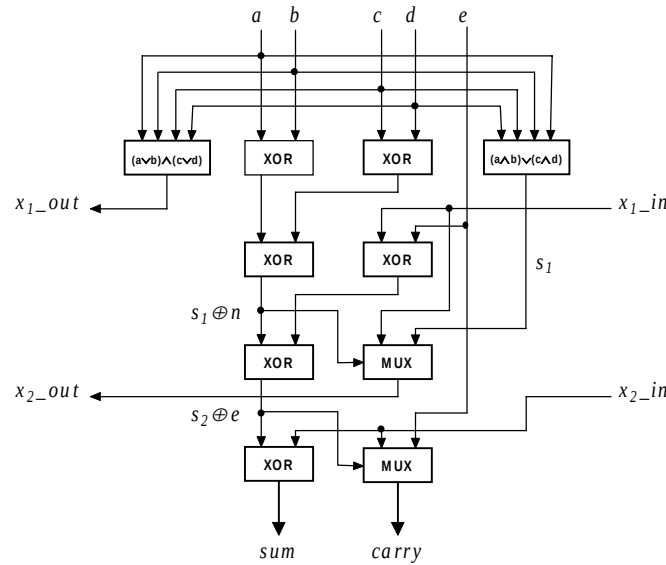


Figure 3. A proposed 5:2 compressor logical decomposition

The 5:2 compressor has not traditionally been used since its straightforward implementation requires $5\Delta_{xor}$ which is equivalent to the delay of fast 6:2 compressor, as found in Song[6]. The fast implementation of the 6:2 compressor uses two 3:2 counters in parallel for 6:4 compression and one 4:2 compressor at the next stage. Using the new fast 5:2 compressor, we can improve the speed of multipliers and fused multiply-adders at specific operand sizes as will be explained in section 3 and section 4.

Before we prove the correctness of the 5:2 compressor logical decomposition, we will define some intermediate variables for the convenience. Table 1 defines a mapping to

represent the value of $a + b + c + d$, where a, b, c and d are input bits having ‘0’ or ‘1’ as their values.

Table 1. A redundant representation of $(a + b + c + d)$

$a + b + c + d$	other conditions	x_{1_out}	n	s_1
0		0	0	0
1		0	1	0
2	$((a \wedge b) \vee (c \wedge d))$ $((a \vee b) \wedge (c \vee d))$	0 1	1 0	1 0
3		1	0	1
4		1	1	1

The following theorem proves the correctness of the 5:2 compressor in figure 3.

Theorem 1 $a, b, c, d, e, x_{1_in}, x_{2_in}$ are input bits of the 5:2 compressors in Figure 3 and $x_{1_out}, x_{2_out}, carry, sum$ are output bits in Figure 3. Then,

$$a + b + c + d + e + x_{1_in} + x_{2_in} = (x_{1_out} + x_{2_out} + carry) \cdot 2 + sum. \quad (1)$$

Proof:

From the definition in table 1, the following holds true.

$$x_{1_out} = ((a \vee b) \wedge (c \vee d)) \quad (2)$$

$$s_1 = ((a \wedge b) \vee (c \wedge d)) \quad (3)$$

$$(s_1 \oplus n) = ((a \oplus b) \oplus (c \oplus d)) \quad (4)$$

$$a + b + c + d = x_{1_out} \cdot 2 + s_1 + n. \quad (5)$$

Let's define x_{2_out} and s_2 as carry and save of $(s_1 + n + x_{1_in})$, respectively. Then,

$$\begin{aligned} x_{2_out} &\equiv (s_1 \wedge n) \vee (s_1 \wedge x_{1_in}) \vee (n \wedge x_{1_in}) \\ &= (s_1 \wedge n) \vee (x_{1_in} \wedge (s_1 \vee n)) \\ &= ((\overline{(s_1 \oplus n)} \wedge s_1) \vee ((s_1 \oplus n) \wedge x_{1_in})) \end{aligned} \quad (6)$$

$$s_2 \equiv ((s_1 \oplus n) \oplus x_{1_in}) \quad (7)$$

$$s_1 + n + x_{1_in} = x_{2_out} \cdot 2 + s_2. \quad (8)$$

In a similar manner, let $carry$ and sum be carry and sum of $(s_2 + e + x_{2_in})$, respectively. Then,

$$carry \equiv ((\overline{(s_2 \oplus e)} \wedge e) \vee ((s_2 \oplus e) \wedge x_{2_in})) \quad (9)$$

$$sum \equiv ((s_2 \oplus e) \oplus x_{2_in}) \quad (10)$$

$$s_2 + e + x_{2_in} = carry \cdot 2 + sum. \quad (11)$$

From the equations 5,8 and 11,

$$\begin{aligned} a + b + c + d + e + x_{1_in} + x_{2_in} &= x_{1_out} \cdot 2 + s_1 + n + e + x_{1_in} + x_{2_in} \\ &= x_{1_out} \cdot 2 + x_{2_out} \cdot 2 + s_2 + e + x_{2_in} \\ &= (x_{1_out} + x_{2_out} + carry) \cdot 2 + sum. \end{aligned} \quad (12)$$

It is readily seen that the block diagram of figure 3 generates $x_{1_out}, x_{2_out}, carry$ and sum according to their definitions in equation 2, 6, 9 and 10 respectively. $\square\square\square$

3: A design example:16-bit x 16-bit MAC design

Figure 4 shows a block diagram of 16-bit x 16-bit 2's complement MAC design. To reduce the number of partial products, radix-4 Booth recoding is used. The partial products and accumulator output are combined to form a PPM(Partial Product Matrix) and undergo 10:2 reduction process. The final two rows after the final reduction process are added in the carry-propagating adder and the accumulator output is fed back for the next accumulation operations.

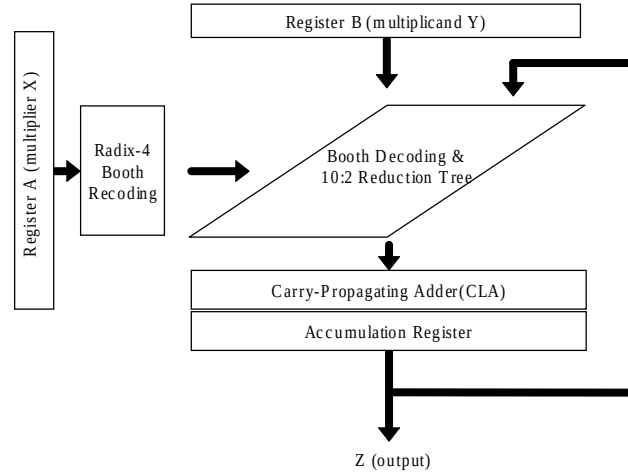


Figure 4. Block diagram of a 16-bit x 16-bit MAC

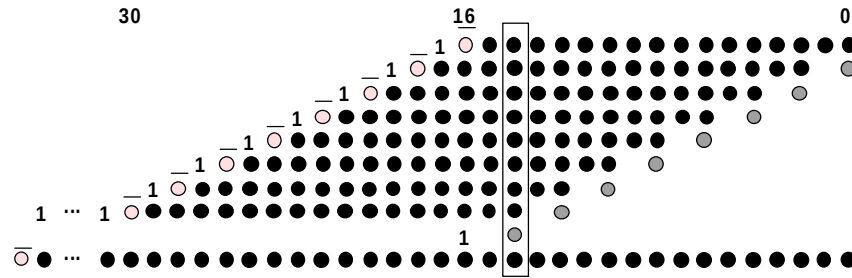


Figure 5. Dot diagram of a 16-bit x 16-bit 2's complement MAC

Figure 5 shows the partial product matrix of the design. In the figure, the sign bits of partial products are shown as white dots and carry-in's of partial products are shown as grey dots. A bar above a dot means the logical inverse of the dot. When a recoded digit is negative, the corresponding partial product has a carry-in. Efficient methods handling 2's complement partial products in PPM(Partial Product Matrix) are well discussed in [14][15][16]. The partial product matrix consists of 8 partial products obtained using radix-4 Booth recoding technique and an accumulator output of the previous stage. However, the maximum height of bit slice is 10 since there are carry-ins with partial products due to the possible negative recoded digits in the Booth recoding process.

Figure 6 shows the necessary 10:2 reduction bit slice obtained using two 5:2 compressors

and one 4:2 compressor. In overall, it takes $7\Delta_{xor}$ compared to $8\Delta_{xor}$ of conventional design using 4:2 compressors and 3:2 counters, which is 14% speed improvement in this design example when we use the new 5:2 compressors.

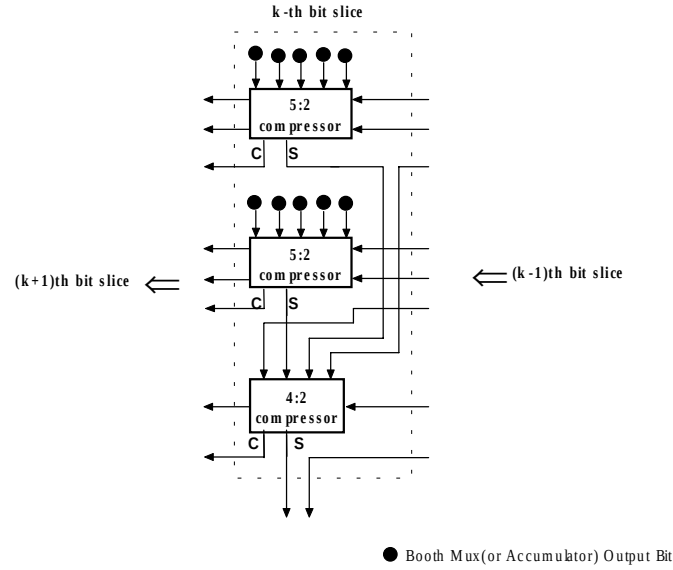


Figure 6. Bit slice for 10:2 reduction

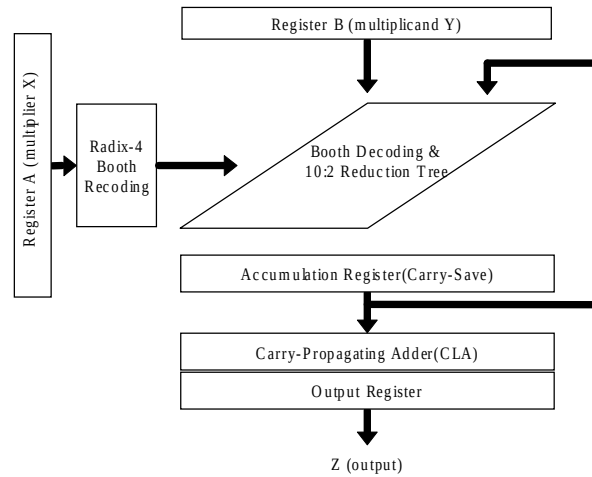


Figure 7. Block diagram using carry-save accumulator output

To improve the throughput of MAC, the accumulator output can be fed back directly in a redundant form(carry-save) as shown in figure 7. In this manner, the delay of carry-propagating adder is separated from the delay of partial product generation and reduction. However, this method entails more register cells. Figure 8 shows the partial product matrix in this case.

Originally, it consists of 9 rows from the radix-4 Booth recoding and two accumulator output rows, which are available at the beginning of the cycle. The carry-in bit of the

last partial product and constant one's are also available without any delay in the cycle. Therefore, these three rows can be reduced to two rows using a row of full adders simultaneously during the partial products generation. The partial product generation requires one Booth encoding delay as well as one Booth decoder(5:1 MUX) delay which will take longer than one full adder delay. With the early row of full adders, there are only 10 rows to be reduced in the main reduction process. Thus, we can apply the 10:2 compression bit slice in the same way as previously discussed.

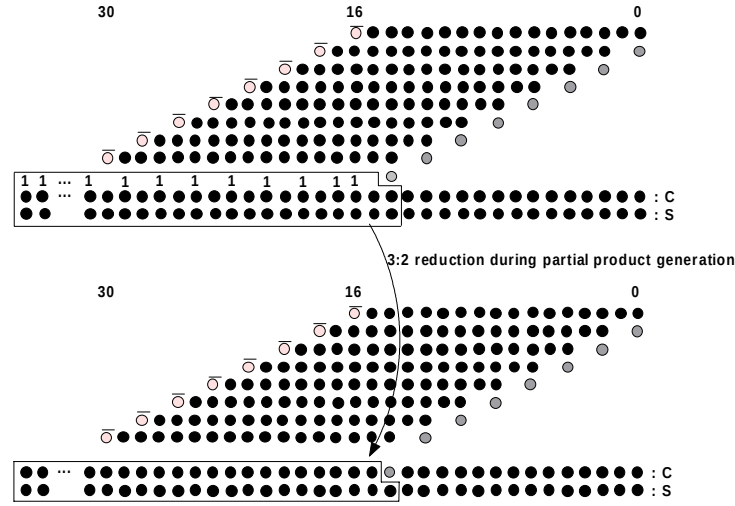


Figure 8. Partial product matrix for a higher throughput MAC

4: Discussion

In this paper, the new 5:2 compressor was applied to a 16-bit x 16-bit MAC design and it was shown that it effectively reduces one Δ_{xor} in the partial product reduction process for 10:2 compression. In general, the 5:2 compressor can be used for other multiplication or multiply-addition applications. The new 5:2 compressor is effective for 5:2,10:2,15:2 and 20:2 compressions. In all these cases, the new 5:2 compressor can eliminate one Δ_{xor} in partial product reduction process, compared to that of the case when only 3:2 counter and/or 4:2 compressor are used.

In the second 16-bit x 16 bit MAC design example, a row of full adders was used for 3:2 reduction for three rows not driven by the Booth recoder in the partial product matrix prior to the main reduction process. The delay of the full adder row is smaller than that of partial product generation when we use radix-4 or higher recoding techniques. In these recoding techniques, Booth encoding and Booth multiplexing are needed for the partial product generation.

If the delay of partial product generation is greater or equal to that of a 4:2 compressor, we can apply 4:2 compression for the early available 4 rows in the partial product matrix. Figure 9 shows an example of this case, which is useful to reduce one more row in the partial product matrix before partial product process begins. The example is for 13-bit x 13-bit 2's complement MAC design. Using this method, the delay become 6 Δ_{xor} for

8:2 reduction instead of 7 Δ_{xor} for 10:2 reduction. This method is applicable when the operand width of the MAC is an odd number.

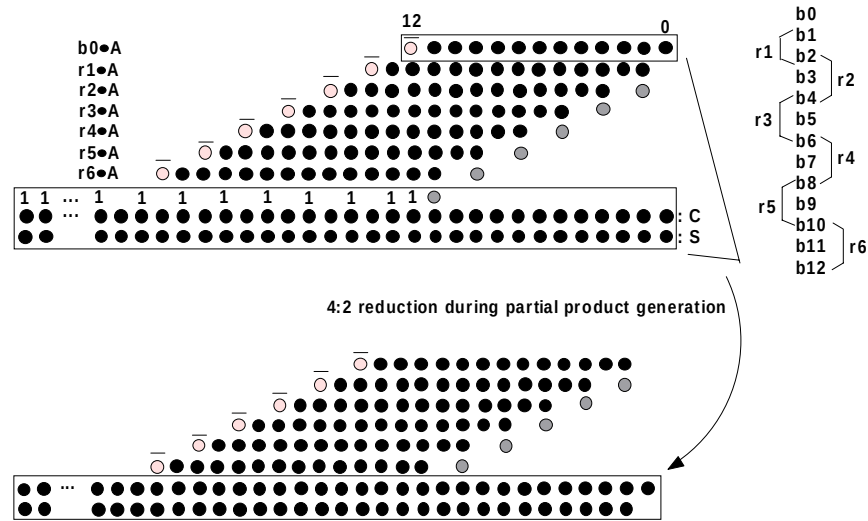


Figure 9. Partial product matrix for a 13-bit x 13-bit 2's complement MAC

5: Conclusions

In this paper, a new logical decomposition is derived for a fast 5:2 compressor which can be used for some multiplier or multiply-addition implementations. The use of 5:2 compressors is well suited to efficient 16-bit x 16-bit MAC designs, where one Δ_{xor} is reduced in the partial product reduction process. This leads to a 14% latency improvement measured in XOR unit delays. In addition, when the accumulator output is left in the carry-save form, we can eliminate one row in partial product matrix before the reduction process begins. These new methods are applied to a 16-bit x 16-bit 2's complement MAC(multiplier and accumulator) design.

Acknowledgments

The authors would like to acknowledge Nobuo Kojima and Jeff Burns at IBM Austin Research Lab for their helpful support.

References

- [1] C. S. Wallace, "A Suggestion For a Fast Multiplier," *IEEE Trans. Electronic Computers*, vol. EC-13, pp. 14-17, 1964.
- [2] L. Dadda, "Some Schemes for Parallel Multiplier," *Alta Freq.*, vol. 34, pp. 349-356, 1965.
- [3] O. L. MacSoley, "High Speed Arithmetic in Binary Computers," *Proc. IRE*, vol. 49, pp. 67-91, 1961.

- [4] H. Sam and A. Gupta, "A Generalized Multibit Recoding of Two's Complement Binary Numbers and Its Proof with Application in Multiplier Implementation," *IEEE Trans. on Computers*, vol. 39, pp. 1006-1015, 1990.
- [5] S. Vassiliadis, E. M. Swartz, and D. J. Hanrahan, "A General Proof for Overlapped Multiple-Bit Scanning Multiplications," *IEEE Trans. on Computers*, vol. 38, pp. 172-183, 1989.
- [6] P. J. Song and G. D. Micheli, "Circuit and Architecture Trade-offs for High-Speed Multiplication," *IEEE Journal of Solid-state Circuits*, vol. 26, pp. 1184-1198, 1991.
- [7] V. G. Oklobdzija, D. Villeger, and S. S. Liu, "A Method for Speed Optimized Partial Product Reduction and Generation of Fast Parallel Multipliers Using an Algorithmic Approach," *IEEE Trans. on Computers*, vol. 45, pp. 294-305, 1996.
- [8] K. Yano et. al., "A 3.8-ns CMOS 16x16-b Multiplier Using Complementary Pass-Transistor Logic," *IEEE Journal of Solid-state Circuits*, vol. 25, pp. 388-395, 1990.
- [9] A. Parameswar, H. Hara, and T. Sakurai, "A Swing Restored Pass-Transistor Logic-Based Multiply and Accumulator Circuit for Multimedia Applications," *IEEE Journal of Solid-state Circuits*, vol. 31, pp. 804-809, 1994.
- [10] M. Izumikawa et. al., "A 0.25- μ m CMOS 0.9-V 100-MHz DSP Core," *IEEE Journal of Solid-state Circuits*, vol. 32, pp. 52-61, 1997.
- [11] N. Ohkubo et. al., "A 4.4 ns CMOS 54 x 54-b Multiplier Using Pass-Transistor Multiplexer," *IEEE Journal of Solid-state Circuits*, vol. 30, pp. 251-257, 1993.
- [12] C. F. Law, S. S. Rofail, and K. S. Yeo, "A Low-Power 16 x 16-b Parallel Multiplier Utilizing Pass-Transistor Logic," *IEEE Journal of Solid-state Circuits*, vol. 34, pp. 1395-1399, 1999.
- [13] J. Mori et. al., "A 10-ns 54 x 54-b Parallel Structured Full Array Multiplier with 0.5- μ m CMOS technology," *IEEE Journal of Solid-state Circuits*, vol. 26, pp. 600-606, 1989.
- [14] C. R. Baugh and B. A. Wooley, "A Two's Complement Parallel Array Multiplications Algorithm," *IEEE Trans. Computers*, vol. C-22, pp. 1045-1047, 1973.
- [15] S. Vassiliadis, E. M. Swartz, and B. M. Sung, "Hard-Wired Multipliers with Encoded Partial Products," *IEEE Trans. on Computers*, vol. 40, pp. 1181-1197, 1991.
- [16] K. A. C. Bickerstaff, M. Schulte, and E. E. Swartzlander, "Reduced Area Multipliers," *Proceedings., International Conference on Application-Specific Array Processors*, pp. 478-489, 1993.
- [17] Neil H. E. Weste and Kamran Eshraghian, *Principles of CMOS VLSI Design: A Systems Perspective*, 2nd ed., Santa Clara, CA, Addison-Wesley Publishing Company, 1993.